

Educação Profissional Paulista

Técnico em
**Ciência de
Dados**

Estrutura de controle de fluxo

Loop "for"

AULA 1

Código da aula: [DADOS]ANO1C2B1S7A1

Exposição



Objetivos da aula

- Conhecer a estrutura de controle **for** e a função **range**.



Competências da unidade (técnicas e socioemocionais)

- Ser proficiente em linguagens de programação para manipular e analisar grandes conjuntos de dados;
- Identificar e analisar problemas, desenvolver alternativas e implementar soluções eficazes durante a execução de um projeto.



Recursos didáticos

- Acesso ao laboratório de informática e/ou internet.



Duração da aula

50 minutos

Loop "for"

Do inglês, podemos traduzir **for** como **para**.

Em Python, o **for** é uma estrutura de controle usada para iterar (percorrer) sequências de elementos, como listas, tuplas, *strings*, dicionários e outros tipos de coleções.

Ele permite executar um bloco de código repetidamente para cada elemento na sequência, tornando mais conveniente e eficiente o processo de percorrer e manipular os elementos.



Tome nota

A sintaxe básica do **for** em Python é a seguinte:

```
for elemento in sequencia:  
    # Bloco de código a ser executado para cada elemento
```

Reprodução – Jupyter Notebook.

Loop "for"

```
for elemento in sequencia:  
    # Bloco de código a ser executado para cada elemento
```

Reprodução - Jupyter Notebook.

Em que:

- **elemento** é uma variável que assume o valor de cada elemento na sequência a cada iteração (é criada de forma automática, portanto, não é preciso defini-la previamente);
- **sequência** é a coleção de elementos pela qual se deseja iterar (percorrer);
- **bloco de código:** é o "to do", o corpo do *loop*.

Exemplos de objetos iteráveis:

- **string** (percorrer os "caracteres do texto");
- **lista** (percorrer cada elemento da lista);
- **tuplas** (percorrer cada elemento imutável da tupla);
- **set** (percorrer cada elemento de conjuntos que não mantêm a ordem de seus itens);
- **dict** (dicionários são iteráveis por chave e itens)
- **Frozenset** (percorrer elementos de um frozenset).

Loop "for"

Podemos usar o **for** para percorrer uma lista de números e imprimir cada número.

Nesse caso, por exemplo, o bloco de código dentro do **for** será executado cinco vezes, sendo uma para cada número na lista `numeros`.

```
1 numeros = [1, 2, 3, 4, 5]
2 for numero in numeros:
3     print(numero)
```

```
1
2
3
4
5
```

Reprodução - Jupyter Notebook.



Tome nota

O **for** é uma ferramenta poderosa para realizar tarefas repetitivas e processar coleções de dados de forma eficiente em Python.

Loop "for"

Sequência iterável **string**:

```
1 for elemento in "Ciência de Dados": # string com 16 caracteres (16 elementos)
2     print(elemento)
```

C
i
ê
n
c
i
a

d
e

D
a
d
o
s

Reprodução – Jupyter Notebook.

Loop "for"

Sequência iterável **lista**:

```
1 for elemento in ['Renata', 'Jones', 'Carol', 'Luis Felipe']: # lista com 4 elementos (strings)
2     print(elemento)
```

```
Renata
Jones
Carol
Luis Felipe
```

```
1 for elemento in [-5, 1, -6, 3, -7, 5]: # lista com 6 elementos (números)
2     print(elemento)
```

```
-5
1
-6
3
-7
5
```

Reprodução – Jupyter Notebook.

Loop "for": exemplo

Sequência iterável **lista**:

```
1 for elemento in ['Palavra', 3, 7.5, True]: # lista com elementos do tipo string, int, float, bool
2     print(elemento)
3     print(type(elemento))
```

```
Palavra
<class 'str'>
3
<class 'int'>
7.5
<class 'float'>
True
<class 'bool'>
```

```
1 for elemento in [[1, 2, 3], [3, 4, 5], [1]]: # lista de lista
2     print(elemento)
3     for item in elemento: # para cada elemento, percorrer a lista -> print em cada item da lista
4         print(item)
```

```
[1, 2, 3]
1
2
3
[3, 4, 5]
3
4
5
[1]
1
```

Reprodução - Jupyter Notebook.

Loop "for": exemplo

Sequência iterável de **tuplas**, **conjunto** e **dicionário**:

```
1 for elemento in ('MG', 'SP', 'RJ'): # tuplas
2     print(elemento)
```

MG
SP
RJ

```
1 for elemento in {'Brasil', 'Paraguai', 'Argentina'}: # set
2     print(elemento)
```

Paraguai
Argentina
Brasil

```
1 for elemento in {'Carolina': 9, 'Renata': 8, 'Luiz Felipe': 8.5}: # dicionário
2     print(elemento)
3
4 # Veremos em outra aula, dic podem ser iterados por items, key
```

Carolina
Renata
Luiz Felipe

Reprodução – Jupyter Notebook.

Loop "for"

Teste para saber se algo é **iterável**:

```
1 # teste para saber se é iterável:  
2 print(iter([1,2,3]))  
3 print(iter('string'))  
4 print(iter((1,2,3)))
```

```
<list_iterator object at 0x000001ECC8DDD3F0>  
<str_iterator object at 0x000001ECC8DDCBE0>  
<tuple_iterator object at 0x000001ECC8DDD3F0>
```

Exemplo de um objeto que **não é iterável**:

```
1 for i in 19:  
2     print(i)
```

```
-----  
TypeError                                Traceback (most recent call last)  
<ipython-input-31-46a84f4128b3> in <module>  
----> 1 for i in 19:  
      2     print(i)  
  
TypeError: 'int' object is not iterable
```

Range

A sintaxe básica da **função range** é a seguinte:

```
range(início, fim, passo)
```

Reprodução – Jupyter Notebook.

Em que:

- **início** é o valor inicial da sequência (por padrão, começa em 0);
- **fim**: é o valor final da sequência (não inclusivo, ou seja, a sequência vai até - 1);
- **passo**: é o incremento entre os números da sequência (por padrão, é 1);
- A função **range** retorna um objeto de sequência que pode ser iterado ou convertido em uma lista, se necessário. **O início e o passo podem ser omitidos.**

Em Python, **range** é uma função embutida que gera uma sequência de números inteiros.

Ele é frequentemente **usado em loops for** para especificar a quantidade de iterações ou para criar listas de números de forma mais eficiente.

Range: exemplo

- iterar de 0 a 4:

```
1 for i in range(5):  
2     print(i)
```

```
0  
1  
2  
3  
4
```

- Iterar de 2 a 10, pulando de 2 em 2:

```
1 for i in range(2, 11, 2):  
2     print(i)
```

```
2  
4  
6  
8  
10
```

Reprodução – Jupyter Notebook.

Range: exemplo

- Criar uma lista de números de 1 a 9:

```
1 numeros = list(range(1, 10))  
2 print(numeros)
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Reprodução - Jupyter Notebook

- O **range** é útil para criar *loops* com um número específico de iterações ou para gerar sequências de números em diversas situações de programação.



Atenção!

Em `range (0,10)`, observe que o valor 10 não entra na lista! O último valor não é incluído.



Vamos
fazer uma
atividade

Esta atividade deverá ser
enviada ao AVA (Ambiente
Virtual de Aprendizagem)



10 minutos



Em grupo

De “while” para “for”

- 1 Calcule a soma dos números de 1 a N, em que N é um número fornecido pelo usuário;
- 2 Usando o *loop* **while**:

```
1 # Solicitar ao usuário um valor N
2 N = int(input("Digite um número inteiro positivo: "))
3
4 soma = 0
5 i = 1
6
7 while i <= N:
8     soma += i
9     i += 1
10
11 print(f"A soma dos números de 1 a {N} é {soma}.")
```

Digite um número inteiro positivo: 5
A soma dos números de 1 a 5 é 15.

Reprodução – Jupyter Notebook

- 3 Use o *loop* **for** para resolver esse exercício.



© Getty Images

O que nós
aprendemos
hoje?

Hoje desenvolvemos

- 1** Conhecimentos sobre o conceito da estrutura de controle **for**.
- 2** Conhecimentos sobre a função embutida **range**.
- 3** A prática da conversão de **while** para **for**.

Saiba mais

ALURA. **Python para Data Science:** primeiros passos. Disponível em:
<https://cursos.alura.com.br/course/python-data-science-primeiros-passos/task/122397>.
Acesso em: 23 jan. 2024.

ALURA. **Python para Data Science:** primeiros passos. Disponível em:
<https://cursos.alura.com.br/course/python-data-science-primeiros-passos/task/123754>.
Acesso em: 23 jan. 2024.

Referências da aula

MENEZES, N. N. C. **Introdução à programação com Python:** algoritmos e lógica de programação para iniciantes. São Paulo: Novatec, 2019.

Identidade visual: imagens © Getty Images

**Educação
Profissional
Paulista**

Técnico em
**Ciência de
Dados**